Phase 2:

Mobile UI Development

Transforming our decision-making logic into a sleek, Material Design Android application using the Flet Framework.

The Flet Architecture

Bridging Python Backend with Flutter Frontend seamlessly for our team of CS & Stats students.

Integration Strategy

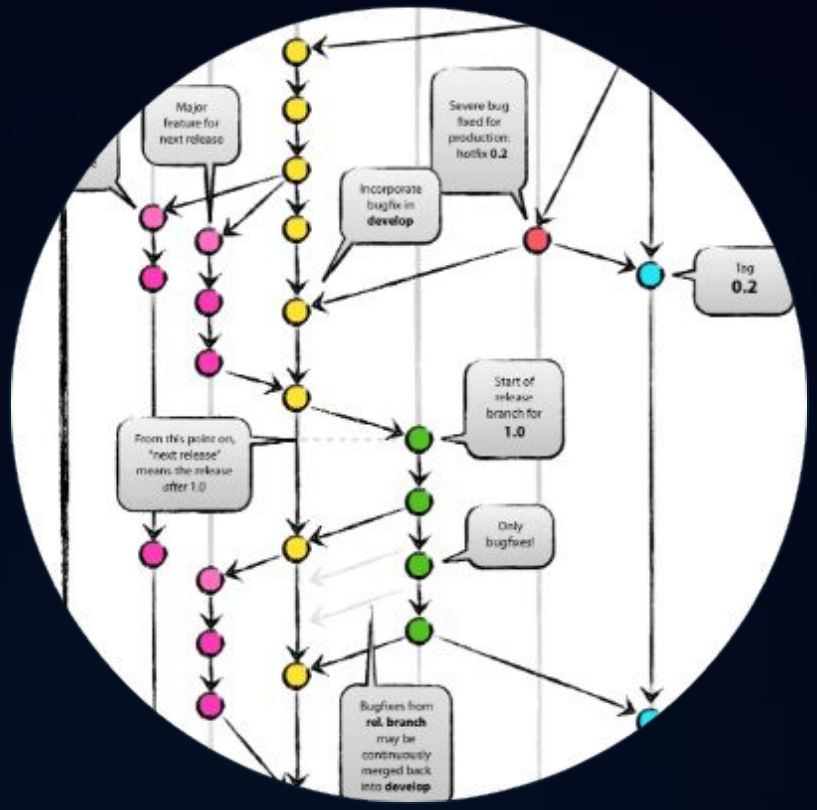
Backend Logic

The core **RandomSelector** class we fixed in the console phase will serve as the "brain," handling all data processing and sampling.

Frontend UI

Flet controls will build the "body," creating an interactive environment where users can input choices and see real-time results.

Team Collaboration



Mastering the Workflow

Each teammate will be assigned a specific "broken" component of the UI. This encourages the use of **Feature Branches** and prevents merge conflicts while teaching Git basics in a real-world scenario.

- Teammate A: UI State Management
- Teammate B: Backend Integration

Intentional Bugs Introduced

The Silent Update

The UI logic updates variables correctly, but the display remains static. **Mission:** Implement page.update() correctly.

The Broken Link

The "Select" button triggers an event, but it's not connected to the backend. **Mission:** Link the UI event to logic methods.

The Dark Void

Text is rendered in a color that matches the background, making it invisible. **Mission:** Master Flet theming and styling.

Modern Design

Mobile-First Aesthetics

Our app leverages Google's Material Design 3. We focus on clean typography, intuitive spacing, and high-contrast elements to ensure accessibility for all users.

This phase teaches students that professional apps aren't just about code, but about user experience (UX) and visual hierarchy.

Development Impact

100%

PYTHON BASED

Zero Frontend Barrier

By using Flet, we bypass the need for JavaScript, CSS, or HTML. This allows our Statistics-focused teammates to focus on **Algorithmic Logic** while still producing a high-quality Android artifact.

UI Best Practices

- **Responsive Layouts:** Using Column and Row to ensure the app fits any screen size.
- **Input Validation:** Guarding the app against empty inputs or invalid counts using snackbars.
- **User Feedback:** Implementing loading icons or distinct result cards.
- **Code Modularity:** Keeping UI code separate from core logic in main.py.

Team Proficiency Growth

The learning curve shows a rapid acceleration as teammates move from abstract console logic to tangible visual results.

Task Allocation Matrix

Teammate	Feature Branch	Core Responsibility	Priority
Teammate A	feat-ui-state	Fix page.update() & Event Handling	High
Teammate B	feat-logic-link	Connect UI to core.logic_selector	High
Team Lead	main	Code Review, Buildozer APK Build	Critical

Project Roadmap

Logic Proof

Console testing complete.

UI Sprint

Fixing introduced bugs.

Refinement

Styling & Error handling.

Deployment

APK Export via Buildozer.

Ready for Mobile?

Next Step: Initialize the android_app/main.py template.

Image Sources

https://nvie.com/img/git-model@2x.png

Source: nvie.com

https://i.ytimg.com/vi/QHWYogOsPx4/maxresdefault.jpg

Source: www.youtube.com

https://png.pngtree.com/thumb_back/fh260/background/20221015/pngtree-abstract-programming-workflow-on-a-screen-while-developing-real-python-code-photo-image_28458240.jpg

Source: pngtree.com